

Dossier projet – RNCP

Développeur d' Application Web

Création du site Web : La plume

Du 23 septembre 2024 au 19 septembre 2025

Développeur D'application Web



Nadège ZONGO



1	Table des matières	
2	Cadrage	4
3	Cahier des charges	4
4	Contenus, arborescence & parcours	5
4.1.1	Arborescence d'écrans (sitemap)	5
5	Contribuer à la gestion d'un projet.....	8
6	Maquetter une application	9
6.1	Charte graphique	9
6.2	Maquettage	12
6.3	Les rôles et les actions de chaque utilisateur	13
7	Architecture multicouche & sécurité	14
8	Développer une interface utilisateur.....	15
9	Développer des composants d'accès aux données	16
9.1	Développer la partie front-end d'une interface utilisateur web	16
9.2	Views Jinja2 :.....	18
9.3	Dynamique UI	19
9.3.1	Formulaires et protection CSRF	19
9.3.2	Catalogue, filtrage et pagination	19
9.3.3	Performance et assets	20
9.4	Mise en place des controllers de routes	21
10	Mettre en place une base de données relationnelle	22
10.1	Diagramme de cas d'utilisation (UML)	22
I.	Développer la partie back-end d'une interface utilisateur web	23

10.2 Dictionnaire de données.....	24
10.3 Modèle Conceptuel de Données (MCD)	26
10.4 Mettre en place une base de données	30
10.5 Création des modèles et migrations	30
10.6 Insertion des données via l'application.....	31
10.7 Journalisation et suivi des opérations	31
10.8 Sauvegardes et sécurité	31
10.9 Avantages de cette mise en place	32
10.10 Développer des composants dans le langage d'une base de données.....	33
10.10.1 Les opérations de CRUD	33
10.10.2 Journalisation NoSQL (MongoDB)	34
11 Préparer et documenter le déploiement d'une application .	36
11.1 Repo GitHub	36
11.2 II. Préparer et documenter le déploiement d'une application	36
12 Déploiement — GitHub Actions → Docker → Railway	39
12.1 Partie GitHub — CI / CD	39
12.2 Partie Docker	39
12.3 Partie Railway	40

Présentation du projet

La Plume est un projet académique réalisé en **équipe**. C'est une boutique en ligne de livres numériques (et audio) qui propose des **histoires personnalisées** pour enfants et adolescents, afin de renforcer la confiance en soi et le plaisir de lire. Le parcours est simple : **découvrir un livre, ajouter au panier, payer en ligne, recevoir le livre par e-mail**. Le site met l'accent sur une **expérience claire et accessible**, des **textes compréhensibles**, et le **respect des données personnelles**.

Objectifs clés

- Offrir des histoires personnalisées et motivantes pour les jeunes.
- Proposer un achat **rapide et rassurant** avec un parcours épuré.
- Garantir l'**accessibilité** (lecture confortable, navigation simple) et le **respect du RGPD**.

Public visé

- Parents et enfants/ados (12–16 ans),
- Personnalisation des contenus (thèmes, prénoms, centres d'intérêt).
- Catalogue évolutif et interface pensée « kids-friendly ».

Organisation

- Projet **en équipe** (cadre scolaire), maquettes validées en **revues hebdo**, suivi via tableau de tâches.

2 Cadrage

Nous avons conduit le cadrage en équipe afin de : clarifier les objectifs (parcours d'achat fluide, première vente test), définir les indicateurs de succès, et borner un MVP centré sur le flux **Découverte, Panier, Paiement, E-mail de confirmation**. Le périmètre et les risques majeurs (paiement, RGPD, accessibilité) ont été identifiés, qualifiés et consignés dans le Kanban pour suivi.

3 Cahier des charges

Nous avons mené un projet académique en équipe de trois pour créer une boutique d'e-books personnalisés, centrée sur un parcours d'achat simple.

Objectif :

proposer des histoires motivantes, avec une expérience fluide et rassurante jusqu'à la première vente test.

Le service s'adresse d'abord aux parents et aux enfants ou adolescents de 12 à 16 ans.

Parcours MVP :

Le parcours minimal comprend l'accueil, la consultation du catalogue avec options de filtrage, l'ouverture d'une fiche livre, l'ajout au panier, le paiement, la page de confirmation, puis l'envoi d'un e-mail de téléchargement.

Maquettage

La conception a suivi une charte graphique et un zoning. Des wireframes et des maquettes Figma ont été produits, puis un prototype cliquable a permis de valider les parcours.

Accessibilité et SEO

Les contrastes ont été vérifiés. La hiérarchie des titres H1, H2 et H3 est respectée. La navigation au clavier est possible et les labels sont explicites. La conformité RGPD est anticipée.

Architecture

L'application est organisée en couches : interfaces, services et données. Cette structuration renforce la robustesse, la sécurité et la maintenabilité.

Données

Le cœur métier repose sur un modèle relationnel couvrant les commandes, les livres et les utilisateurs. Les journaux d'usage sont stockés séparément pour l'analyse.

Intégrations

Le paiement s'appuie sur Stripe en mode test. Un e-mail de confirmation est envoyé après l'achat. Un brouillon d'API au format OpenAPI et des webhooks de paiement sont prévus.

Organisation

Le rythme de travail inclut une réunion quotidienne d'environ trente minutes et des revues hebdomadaires. Le suivi s'effectue sur un Kanban (GitHub ou Trello). La communication passe par Slack, Gmail et Google Meet. L'outillage Docker est en place.

Qualité et mesure

Un plan de tests couvre les niveaux unitaires, l'intégration et l'accessibilité. Une procédure de déploiement est décrite dans un document dédié (deploy.md). Les indicateurs de succès suivent la conversion, l'abandon de panier et le temps nécessaire pour parcourir les étapes clés.

4 Contenus, arborescence & parcours

4.1.1 Arborescence d'écrans (sitemap)

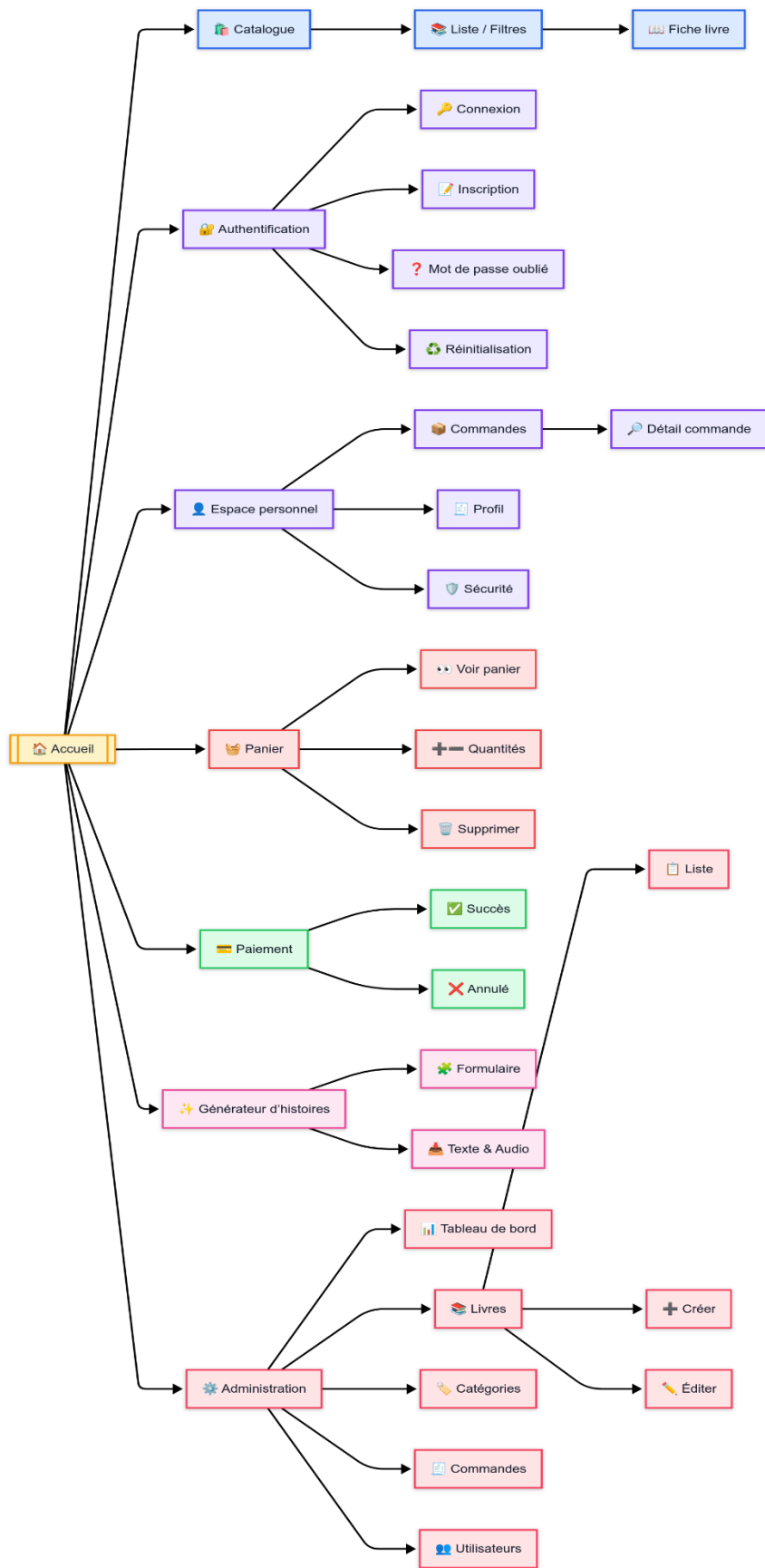
Le visiteur arrive sur la page d'accueil et repère immédiatement les entrées clés : catalogue, générateur d'histoires, panier et connexion. Depuis le catalogue, il parcourt les livres paginés, filtre par catégorie et ouvre une fiche détaillée pour consulter résumé, auteur et prix. Un clic ajoute l'article au panier sans quitter la page ; un indicateur met à jour la quantité et le total.

S'il souhaite finaliser l'achat, le panier présente les lignes, permet d'ajuster les quantités et propose de passer à la commande. La connexion s'effectue alors via e-mail et mot de passe, avec une inscription possible si aucun compte n'existe : le formulaire contrôle le format des champs et évalue la robustesse du mot de passe avant création du compte. Une fois authentifié, l'espace personnel affiche les informations essentielles et l'historique des commandes (date, montant, statut), avec un accès au détail.

Le paiement s'effectue en environnement sécurisé de test ; à la confirmation, la commande est enregistrée et une page de succès clôture le parcours. En parallèle, le générateur d'histoires accessible via /story/generator recueille l'âge, le prénom du héros, le genre, les thèmes et le ton, produit un conte affiché sous forme de flipbook, puis propose le téléchargement en texte (et audio lorsque disponible).

Les pages d'administration, réservées au rôle "admin", centralisent la gestion des catalogues, catégories, utilisateurs et commandes. Tout au long du parcours, les

formulaire valident côté client et côté serveur, les contenus affichés sont échappés, et les informations sensibles restent hors du code grâce aux variables d'environnement. Résultat : un chemin utilisateur clair (accueil → découverte → ajout panier → connexion/inscription → paiement → confirmation) et un back-office strictement contrôlé.

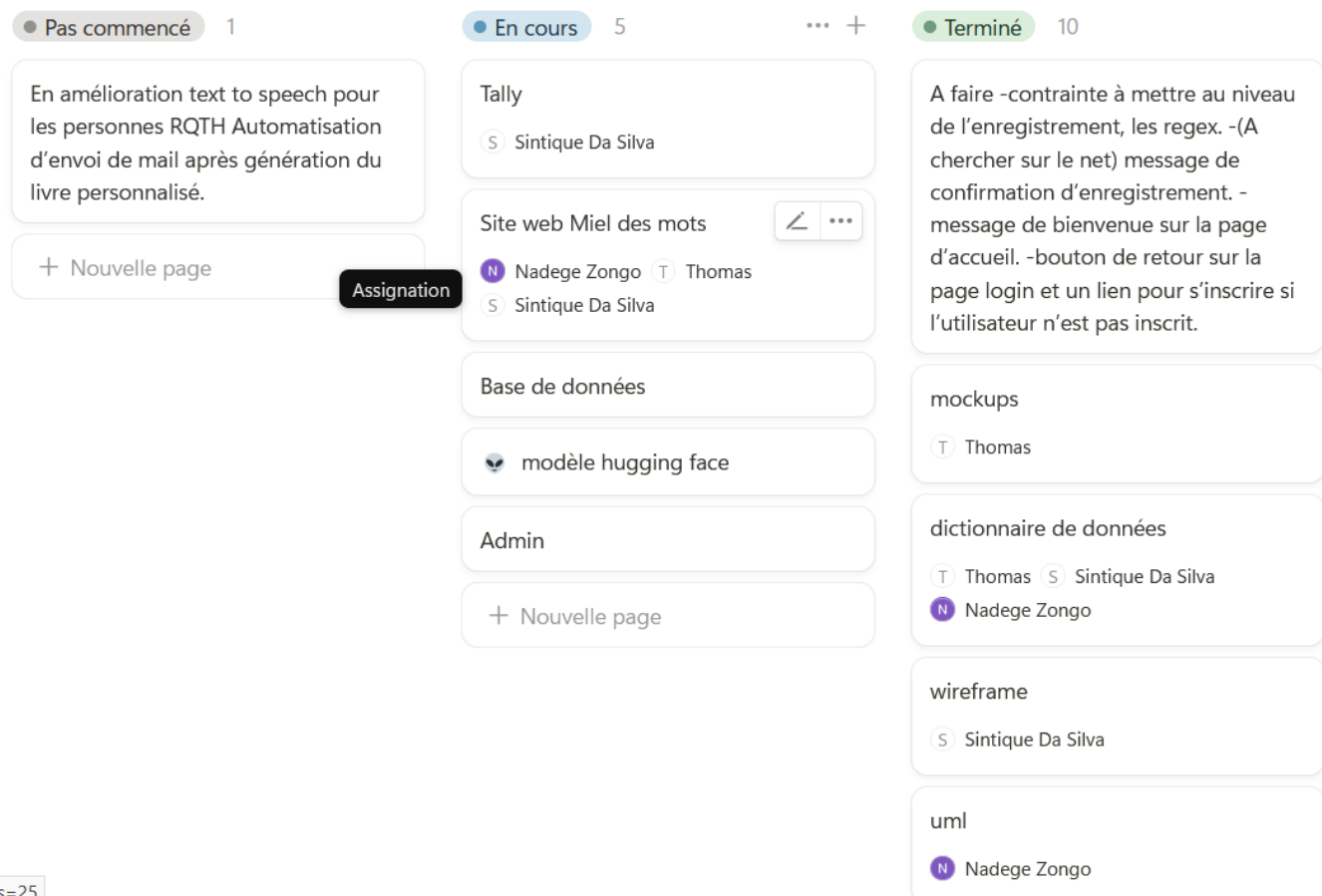


5 Contribuer à la gestion d'un projet

Nous avons conduit le projet en adoptant une méthode agile, avec un fonctionnement itératif et collaboratif. Un tableau Kanban a été utilisé pour planifier, suivre l'avancement et répartir les tâches en trois colonnes (À faire, En cours, Terminé).

Chaque membre de l'équipe avait un rôle attribué afin d'assurer la complémentarité et la fluidité du travail :

Sans titre ...



6 Maquetter une application

Après cela Côté design, nous avons enchaîné **zoning**, **wireframes**, **maquettes Figma**, **prototype cliquable**, en nous appuyant sur un **design system** (palette pastel violet/bleu/gris, composants, états). Les **contrastes texte/fond** sont **conformes WCAG 2.1 (AA/AAA)**, **vérifiés sur des sites d'audit de contraste** (ex. color.co).

Sur cette base, nous avons développé l'interface web avec des **templates Jinja2** modulaires, des **composants UI réutilisables** et un **drawer de panier** responsive.

La **sécurité by design** a été intégrée dès la conception : **identifiant e-mail unique**, **RBAC** (droits par rôle), **secrets Stripe externalisés** via variables d'environnement, **validations client & serveur**, **protection CSRF** (et en-têtes de sécurité adaptés).

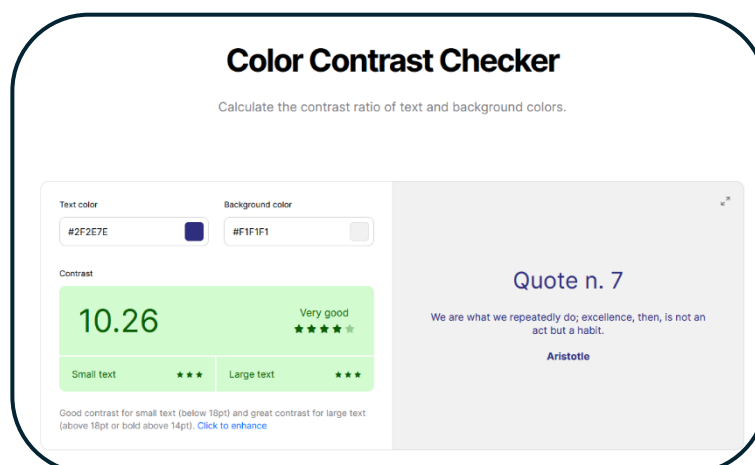
6.1 Charte graphique

Objectif :

J'ai défini une identité visuelle cohérente et durable, puis je l'ai traduite en design tokens et composants réutilisables, afin d'assurer la constance UI et d'accélérer le développement.

Palette & couleurs d'état. :

J'ai construit une palette pastel (violet, bleu et du gris). J'ai documenté pour chaque couleur son usage et ses contraintes de contraste respectant l'accessibilité (RGAA/WCAG)



Typographies :

J'ai défini une échelle typographique claire (H1, H2, H3, corps, petit) avec interlignage et espacements normalisés, et formalisé la hiérarchie des titres (un seul H1 par page, H2 pour les sections, H3 pour les sous-sections), ce qui améliore le référencement naturel (SEO) et l'accessibilité.

Boutons :

J'ai défini trois variantes **Primary**, **Secondary** et **Ghost** avec les états **normal**, **survol (hover)**, **actif**, **désactivé** et **focus** (anneau de focus **2 px**). Les contrastes respectent le **RGAA** et les libellés sont explicites.

Drawer panier (panneau latéral gauche) :

J'ai réalisé un **panneau latéral gauche** utilisable sur **desktop et mobile**, avec **bouton "Fermer"**, **overlay cliquable**, fermeture au **touche Échap**, **piégeage du focus** (focus trap) et **retour du focus** sur le bouton déclencheur.

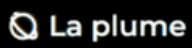
Les images :

Les **images** ont un **texte alternatif** prévu (si utile) ; les icônes décoratives sont ignorées par les lecteurs d'écran.

Formulaires :

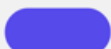
J'ai systématisé l'**association du label au champ**, rendu les **labels obligatoires** visibles, utilisé des **placeholders explicites** (sans remplacer le label) et positionné des **messages d'erreur** proches du champ, **clairs** et

compréhensibles. Les champs requis sont indiqués (*), avec **aides contextuelles** et **validation** côté client et serveur

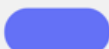


La plume

Nous aimons vous faire sourire



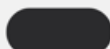
#564BEC



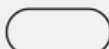
#6271F7



#2F2E7E



#262627



#F1F1F1

Les logos & Les logos alternatifs

 La plume

 La plume

 La plume

La Plume



Polices utilisées

h1 : Poppins

h2: Lato

h3: Roboto

Paragraphe : Open Sans

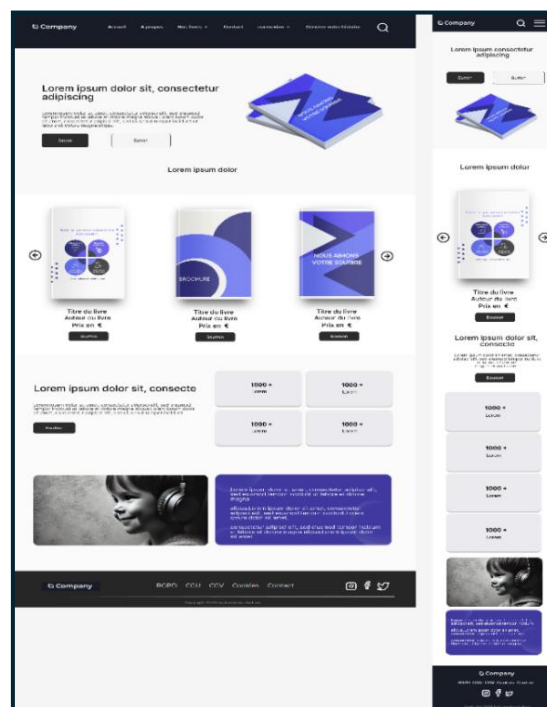
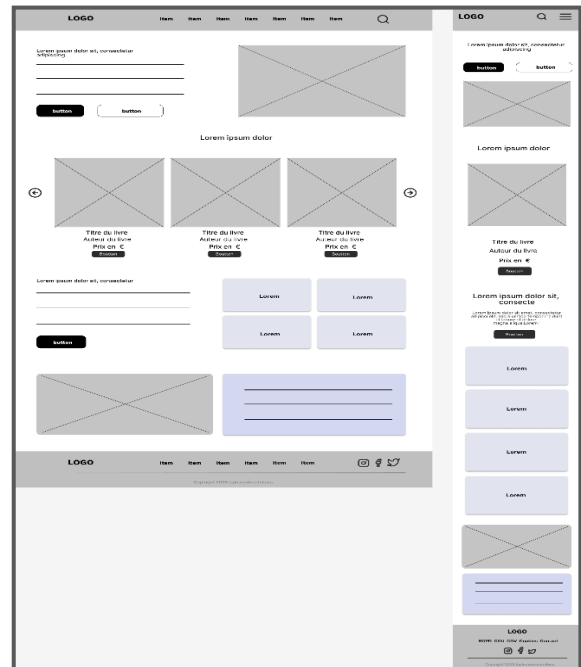
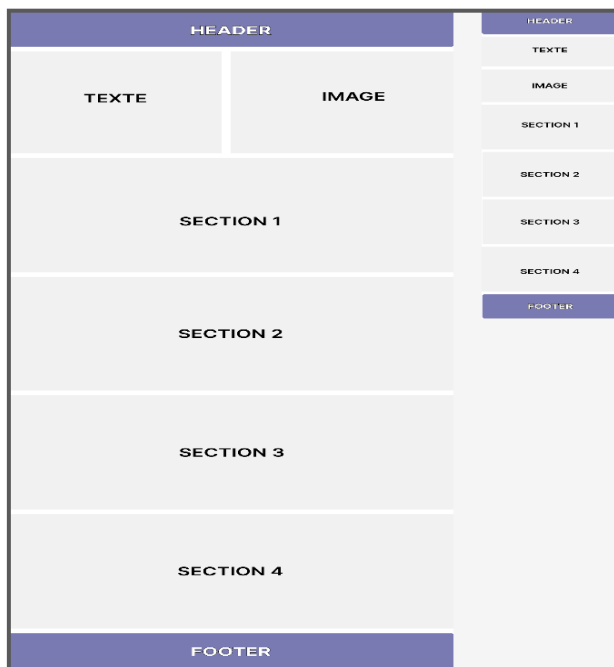
A brand board is a visual guide that outlines how each brand element should be used. It helps your business stay on brand, ensuring consistency across all your designs, whether for print or digital materials.

6.2 Maquettage

Zoning : répartition header, body et footer.

Wireframes : Home, Gallery (filtres, tri, pagination), Détail livre (CTA “Ajouter au panier”), Formulaires (inscription/connexion), drawer panier.

Mockups Figma + prototype cliquable : flux inscription, connexion , page accueil, galerie, filtrage , panier, détail du panier, récap, paiement, confirmation , espace compte.



6.3 Les rôles et les actions de chaque utilisateur

L'application distingue clairement trois profils. Le visiteur, non connecté, parcourt le catalogue, ouvre les fiches livres, ajoute des articles au panier et lance le parcours d'achat ; l'étape de paiement lui demande alors de se connecter ou de créer un compte. Il accède aussi librement au générateur d'histoires via /story/generator, saisit l'âge, le héros et les thèmes, puis lit le conte en flipbook et télécharge le fichier produit.

Une fois authentifié, le client dispose de son espace personnel : l'interface affiche ses informations essentielles et l'historique des commandes, avec accès au détail (date, montant, statut). Le panier reste associé à son compte, le paiement se fait en environnement sécurisé de test, et la confirmation enregistre la commande avant d'afficher la page de succès. La déconnexion est disponible à tout moment depuis le menu.

Le rôle administrateur ouvre un tableau de bord réservé, qui concentre la gestion du catalogue (livres, catégories, auteurs). Les routes sensibles vérifient systématiquement le rôle serveur-side, et toute tentative d'accès sans droit redirige vers la connexion ou la page d'accueil.

Action / Rôle	Visiteur	Client	Admin
Voir l'accueil / catalogue / détails d'un livre	✓	✓	✓
S'inscrire / Se connecter	✓	✓	✓
Générer une histoire (/story/generator, API)	✓	✓	✓
Ajouter au panier	✓	✓	✓
Voir / modifier le panier	✓	✓	✓
Passer commande / payer	✗	✓	✓
Voir l'espace personnel (/account)	✗	✓	✓
Voir ses commandes / détails	✗	✓	✓
Accéder au tableau de bord admin (/admin)	✗	✗	✓
Gérer le catalogue (livres, catégories, auteurs)	✗	✗	✓
Gérer les commandes	✗	✗	✓

Gérer les utilisateurs	X	X	✓
------------------------	---	---	---

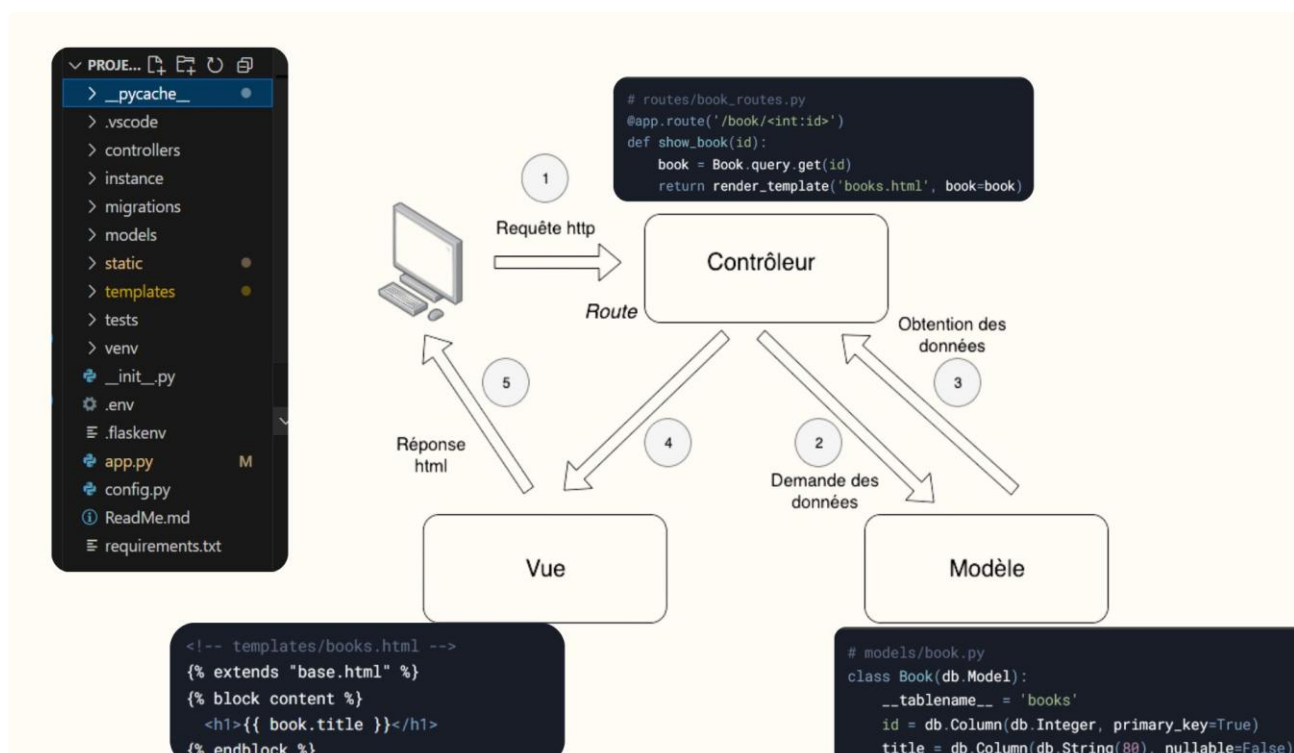
7 Architecture multicouche & sécurité

Décrire l'organisation du code selon le pattern **Modèle–Vue–Contrôleur**, les choix techniques (framework, base de données, ORM, services), et les garanties de sécurité mises en place.

Nous avons adopté une architecture **MVC** structurée en trois niveaux, tout en conservant une séparation claire des responsabilités :

- **Vues (Vue)** : gabarits **Jinja2** et composants UI ; elles affichent les données et appliquent la charte graphique, l'accessibilité (structure H1/H2/H3, focus clavier) et les contraintes UX.
- **Contrôleurs (Controller)** : **Blueprints Flask** qui reçoivent les requêtes, orchestrent le flux applicatif (catalogue, fiche livre, panier, checkout) et appellent les services métier ; gestion centralisée des réponses et des erreurs.
- **Modèles (Modèle)** : **SQLAlchemy** pour le mapping objet–relationnel (utilisateurs, livres, commandes, paiements), avec transactions et requêtes optimisées.

Autour du MVC, une **couche de services métier** encapsule les règles (personnalisation, gestion du panier, calculs, intégration **Stripe** pour le paiement, envoi des e-mails de confirmation) .



8 Développer une interface utilisateur

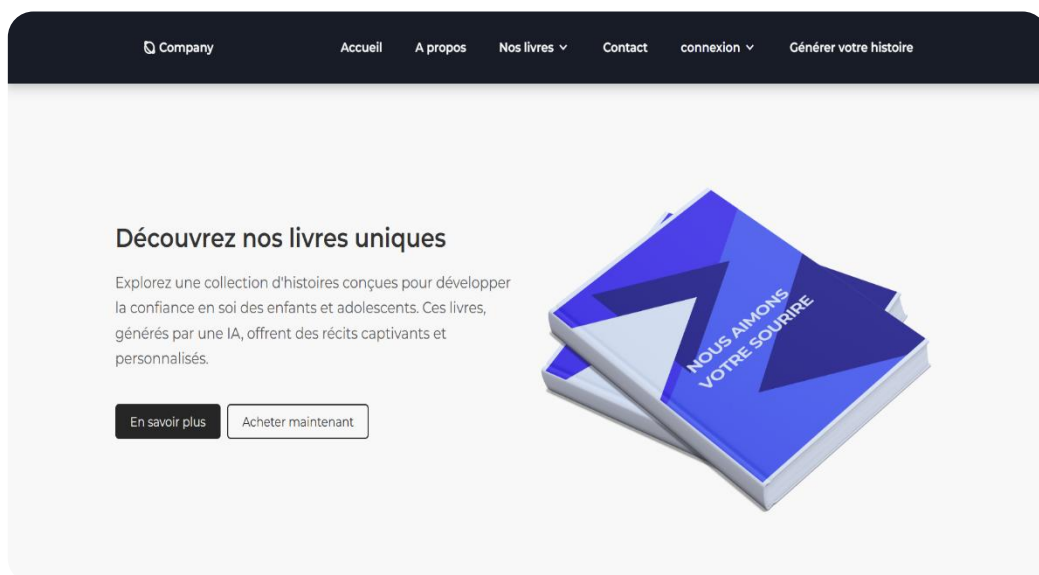
L'interface s'organise autour d'un parcours principal **accueil , catalogue , panier , authentification, paiement, confirmation** et le **générateur d'histoires** accessible directement depuis la navigation.

La vitrine regroupe l'accueil, la liste paginée avec filtres par catégorie et la fiche livre ; l'ajout au panier se fait sans changer de page via un tiroir latéral qui permet d'ajuster les quantités.

L'authentification propose la connexion ou l'inscription et affiche une jauge de robustesse du mot de passe ; une fois connecté, l'espace personnel présente l'historique des commandes avec accès au détail, tandis que les pages d'administration restent limitées au rôle admin (sécurité).

Les écrans gèrent les **états de chargement, vides, erreurs et succès** avec des messages courts et contextualisés, en cohérence avec les validations serveur décrites dans la section Accès aux données .

La **responsivité** s'appuie sur une grille fluide pour mobile et desktop, et l'**accessibilité**




```

248
249 @media (max-width: 360px) {
250   .container {
251     /* fallback + logique */
252     padding-left: 1rem;
253     padding-right: 1rem;
254     padding-inline: 1rem;
255   }
256
257   .nav__link {
258     padding-inline: 1rem;
259   }
260 }

```



9 Développer des composants d'accès aux données

9.1 Développer la partie front-end d'une interface utilisateur web

La partie Front-End reproduit l'apparence et les scénarios de l'application Desktop afin d'offrir une expérience cohérente sur tous les supports. Chaque page métier (accueil, catalogue, détail livre, panier, compte, générateur d'histoires) utilise un **template HTML** dédié. Les **feuilles de style CSS** assurent une identité visuelle homogène (typographies, couleurs, grilles responsives), tandis que des **scripts JavaScript** apportent les interactions clés : ouverture du panier en tiroir, mise à jour des quantités sans rechargement, évaluation visuelle de la robustesse du mot de passe, et affichage « flipbook » des histoires générées. L'ensemble s'appuie sur Flask (rendu côté serveur), garantissant des formulaires clairs et accessibles.

Bienvenue parmi nous

Connectez-vous pour accéder à votre bibliothèque personnelle et découvrir nos nouveautés.



Se connecter

+ Créer un compte

Commencez votre aventure littéraire dès maintenant

Prénom

Nom

Email

Mot de passe

Sécurité: Fort

12 caractères minimum

1 majuscule

1 minuscule

1 chiffre

1 caractère spécial

Téléphone (optionnel)

☒ J'accepte les conditions d'utilisation et la politique de confidentialité

S'inscrire

```

<form action="{{ url_for('register_bp.register') }}" method="POST" id="registerForm">
  <!-- Groupe de champs -->
  <div class="input-group">
    <label for="user_firstname"><i class="fas fa-user"></i> Prénom</label>
    <input type="text" id="user_firstname" name="user_firstname" placeholder="Votre prénom" required minlength="2">
  </div>

  <div class="input-group">
    <label for="user_lastname"><i class="fas fa-user"></i> Nom</label>
    <input type="text" id="user_lastname" name="user_lastname" placeholder="Votre nom de famille" required minlength="2">
  </div>

  <div class="input-group">
    <label for="user_email"><i class="fas fa-envelope"></i> Email</label>
    <input type="email" id="user_email" name="user_email" placeholder="exemple@email.com" required>
  </div>

  <!-- Champ mot de passe avec validation -->
  <div class="input-group password-group">
    <label for="user_password"><i class="fas fa-lock"></i> Mot de passe</label>
    <div class="password-input-container">
      <input type="password" id="user_password" name="user_password" placeholder="Créez un mot de passe" required minlength="12">
      <button type="button" class="toggle-password" aria-label="Afficher le mot de passe">
        <i class="fas fa-eye"></i>
      </button>
    </div>
  </div>

```

9.2 Views Jinja2 :

Un gabarit principal `base.html` définit l'en-tête, la navigation, le pied de page, ainsi que des blocs structurants comme `title`, `head_extra`, `content` et `scripts`. Chaque page étend ce layout afin de garantir une mise en page cohérente et de limiter la duplication. Les éléments transverses (barre de navigation, messages flash, modale panier) sont inclus depuis `partials/` pour centraliser leur maintenance.

```
<!doctype html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <title>{% block title %}La Plume{% endblock %}</title>
    {% block head_extra %}{% endblock %}
  </head>
  <body>
    {% include "partials/navbar.html" %}
    <main id="main" class="container">
      {% include "partials/flash.html" %}
      {% block content %}{% endblock %}
    </main>
    {% include "partials/footer.html" %}
    {% block scripts %}{% endblock %}
  </body>
</html>
```

9.3 Dynamique UI

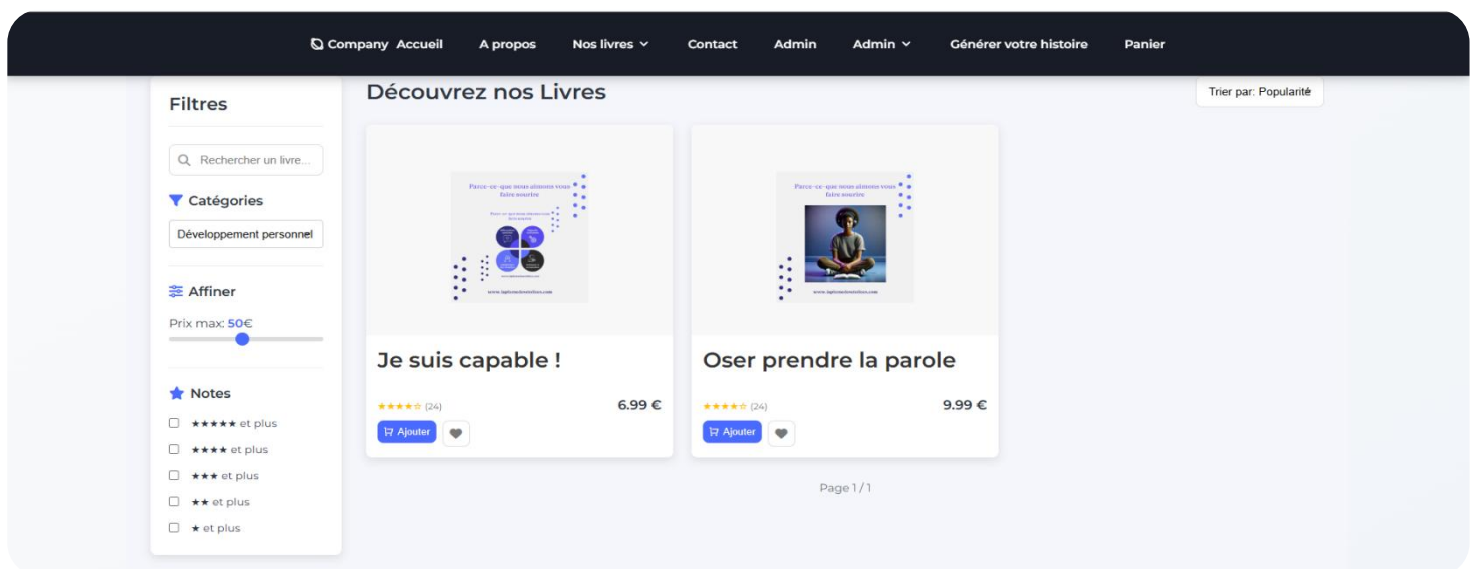
Afin de rendre les écrans interactifs, j'ai implémenté la validation des formulaires (inscription/connexion), le filtrage/tri/pagination dans la galerie, et le drawer panier (ouverture, fermeture, focus clavier). Les messages d'erreur sont placés près des champs et restent compréhensibles.

9.3.1 Formulaires et protection CSRF

Les formulaires incluent systématiquement un token CSRF et affichent les erreurs de validation au plus près des champs. Les valeurs saisies sont préservées en cas d'erreur de soumission pour éviter toute frustration utilisateur. Les contraintes d'accessibilité sont respectées avec des labels liés aux champs, des indications d'erreur via aria-describedby et une annonce des messages par une région aria-live.

9.3.2 Catalogue, filtrage et pagination

Le catalogue permet de filtrer les livres par catégorie et par prix. Les résultats s'affichent page par page pour garder le site rapide. À chaque changement de page, l'information est annoncée aux lecteurs d'écran et le focus clavier revient en haut de la liste pour faciliter la navigation.



9.3.3 Performance et assets

Les assets statiques sont servis via `url_for('static', filename='...')` pour profiter du cache navigateur. Les blocs scripts sont chargés en fin de page et, lorsque possible, en mode différé pour préserver la réactivité initiale. Les composants critiques au dessus de la ligne de flottaison restent légers pour réduire le Time to Interactive.

Content de vous revoir

Connectez-vous pour accéder à votre compte

Email



Veuillez saisir une adresse e-mail valide.



[Mot de passe oublié ?](#)

Se connecter

ou continuer avec



Google



Facebook

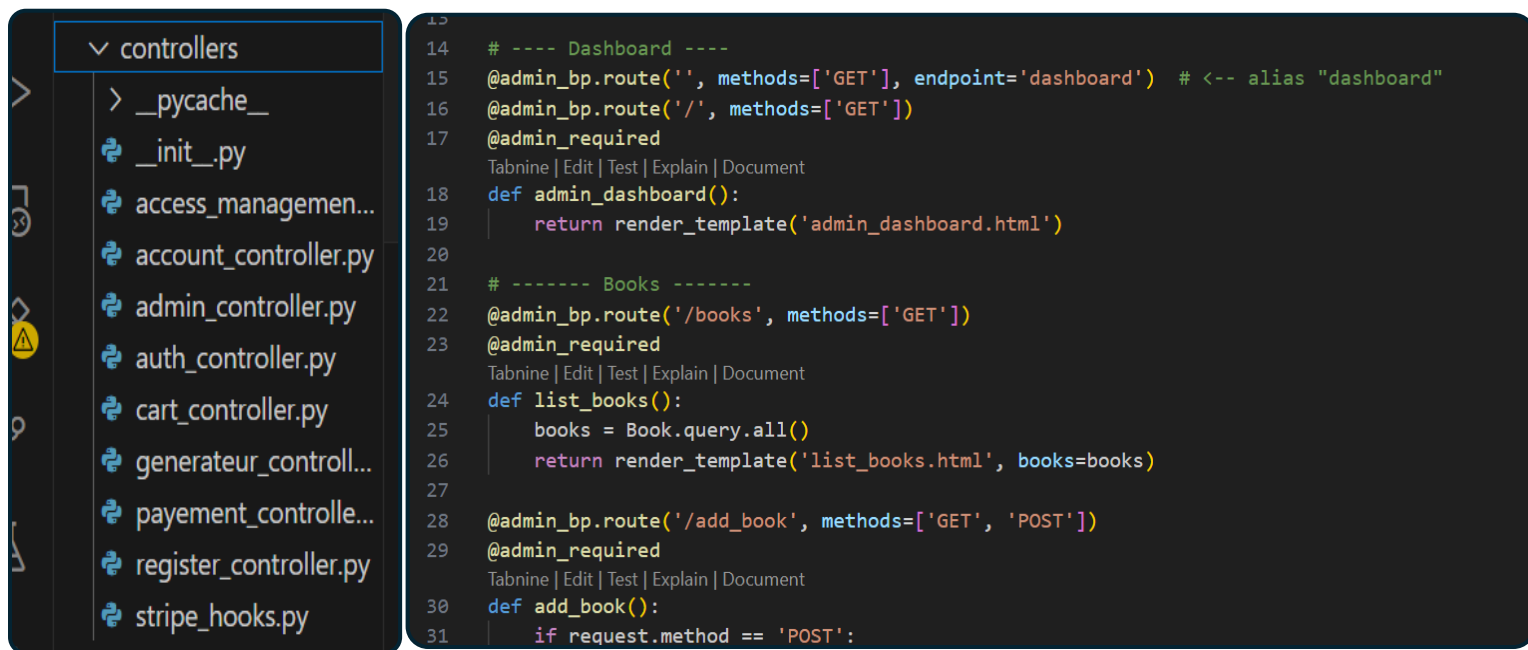
Nouveau ici ?

Créez un compte pour découvrir notre collection exclusive

S'inscrire

9.4 Mise en place des controllers de routes

Dans la partie Controllers toutes les routes (URLs) disponibles, permettent de naviguer vers les différentes pages du site. Chaque route est associée à une fonction spécifique du contrôleur, qui traite les requêtes entrantes, récupère les données nécessaires, et transmet les informations appropriées à la vue pour affichage. Ce fichier importe donc les bibliothèques nécessaires et répertorie toutes les routes (URLs) possibles qui nous emmènent sur les diverses pages du site.



```
13
14 # ---- Dashboard ----
15 @admin_bp.route('/', methods=['GET'], endpoint='dashboard') # <-- alias "dashboard"
16 @admin_bp.route('/', methods=['GET'])
17 @admin_required
18 def admin_dashboard():
19     return render_template('admin_dashboard.html')
20
21 # ----- Books -----
22 @admin_bp.route('/books', methods=['GET'])
23 @admin_required
24 def list_books():
25     books = Book.query.all()
26     return render_template('list_books.html', books=books)
27
28 @admin_bp.route('/add_book', methods=['GET', 'POST'])
29 @admin_required
30 def add_book():
31     if request.method == 'POST':
```

```
106
107 # ---- Blueprints
108 app.register_blueprint(register_bp, url_prefix="/register")
109 app.register_blueprint(login_bp, url_prefix="/auth")
110 app.register_blueprint(admin_bp, url_prefix="/admin")
111 app.register_blueprint(cart_bp, url_prefix="/cart")
112 app.register_blueprint(payement_bp, url_prefix="/payement")
113 app.register_blueprint(account_bp, url_prefix="/account")
114
```

10 Mettre en place une base de données relationnelle

Pour représenter qui fait quoi dans le système, nous distinguons trois acteurs : le **visiteur** (qui n'a pas de compte), l'**utilisateur** authentifié et l'**administrateur** qui gère les comptes et le contenu. Un **diagramme de cas d'utilisation UML** permet de visualiser leurs objectifs et leurs interactions avec l'application, à l'intérieur de la frontière du système La Plume . On y voit par exemple : consulter et filtrer le catalogue, créer un compte, se connecter, ajouter au panier, payer, télécharger son livre, ainsi que les actions d'administration comme gérer le catalogue et les comptes.

Ce diagramme sert de support commun pour aligner l'équipe sur le périmètre fonctionnel, prioriser le MVP et préparer les scénarios de test.

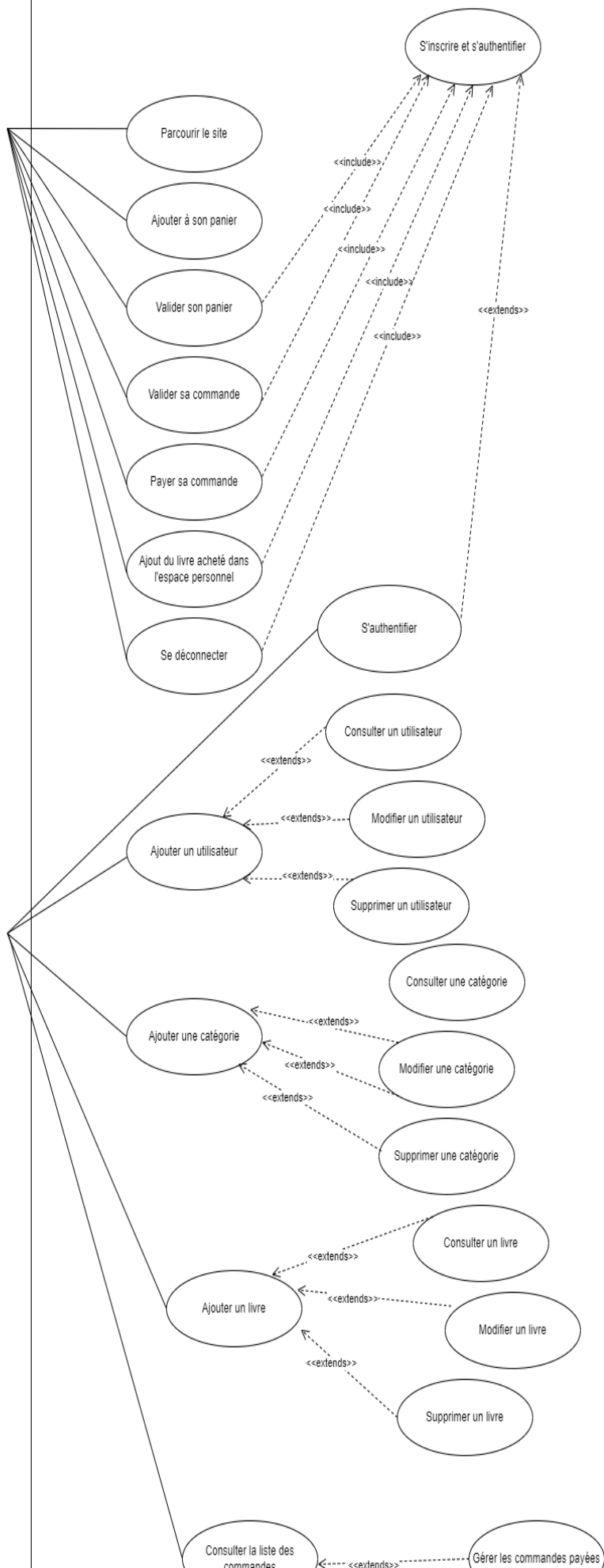
10.1 Diagramme de cas d'utilisation (UML)

Miel des mots

Visiteur



Admin



- **Routes HTTP :**

Pour la modélisation de la base de données de La Plume, j'ai utilisé Mocoodo, Workbench et draw.io permettant de réaliser des schémas clairs et hiérarchisés selon le modèle MERISE.

Ce modèle a été retenu pour sa capacité à représenter de manière progressive la structure de la base, depuis la vision conceptuelle jusqu'à l'implémentation physique, tout en restant adaptable à l'évolution du projet.

10.2 Dictionnaire de données

J'ai mis en place un dictionnaire des données sert à décrire clairement et précisément toutes les données stockées dans la base, pour que toute personne (développeur, administrateur, auditeur, etc.) puisse comprendre la signification, l'usage et les contraintes de chaque champ. Ici nous avons une représentation de quelques tables. Le reste des tables seront dans la partie annexes

Table : User

Nom du champ	Description	Clé primaire	Obligatoire	Valeurs par défaut	Valeurs autorisées
user_id	Identifiant unique pour chaque utilisateur	Oui	Unique, non nul		
user_firstname	Prénom de l'utilisateur	Non	Non nul		
user_lastname	Nom de famille de l'utilisateur	Non	Non nul		
user_email	Adresse email de l'utilisateur	Non	Non nul, unique		Format d'email valide
user_signup_date	Date d'inscription de l'utilisateur	Non	Non nul		
user_password	Mot de passe crypté pour l'authentification	Non	Non nul		Lettres, chiffres, caractères spéciaux
user_phone	Numéro de téléphone de l'utilisateur	Non	Non nul		Format valide (Regex)



Table : Orders

Nom du champ	Description	Clé primaire	Obligatoire	Valeurs par défaut	Valeurs autorisées
order_id	Identifiant unique pour chaque commande	Oui	Unique, non nul		
order_date	Date à laquelle la commande a été passée	Non	Non nul		
user_id	Identifiant de l'utilisateur ayant passé la commande	Clé étrangère	Non nul		Référence user_id dans la table User
status	Statut de la commande (ex. "en attente", "expédié")	Non	Non nul	"pending"	"pending", "shipped", "cancelled"

10.3 Modèle Conceptuel de Données (MCD)

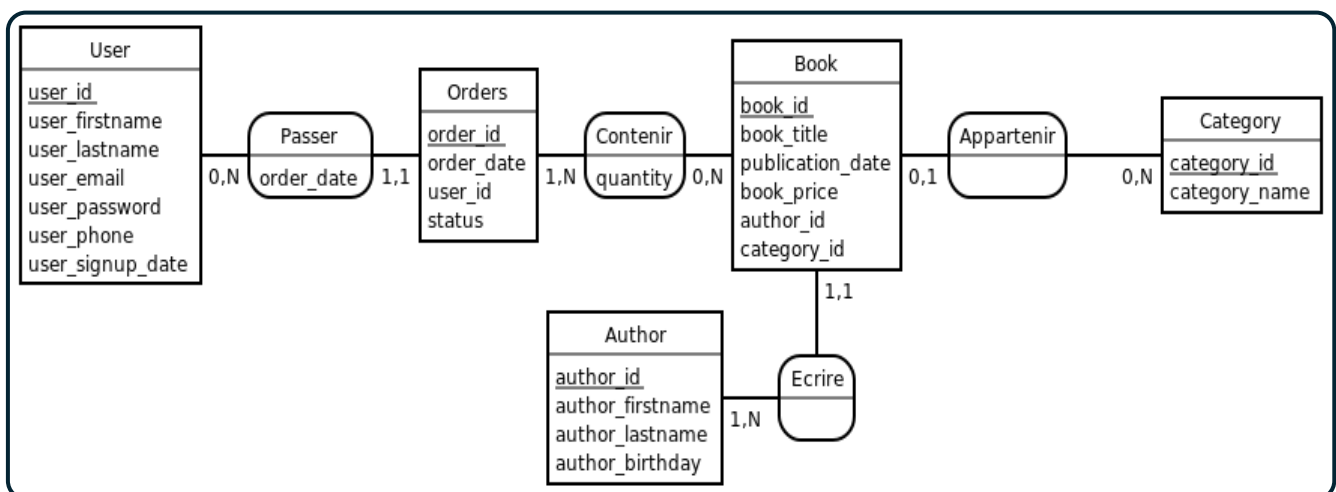
Le MCD a permis d'identifier les **entités principales** et leurs **relations** :

- **User** : représente les clients et les administrateurs du site.
- **Book** : stocke les informations des livres (titre, auteur, prix, résumé, fichier numérique).
- **Author** : conserve les données relatives aux auteurs.
- **Category** : regroupe les livres par thématiques.
- **Order** : enregistre les commandes passées par les utilisateurs.
- **Order_Details** : détaille les articles contenus dans chaque commande.
- **Payment** : conserve les informations liées aux transactions Stripe (statut, montant, date).

Les associations identifiées permettent de représenter :

- Un **utilisateur** peut passer plusieurs **commandes**.
- Une **commande** contient plusieurs **livres** via **Order_Details**.
- Un **livre** est écrit par un **auteur** et appartient à une ou plusieurs **catégories**.

Ce schéma abstrait a permis de valider les besoins fonctionnels avant toute implémentation.

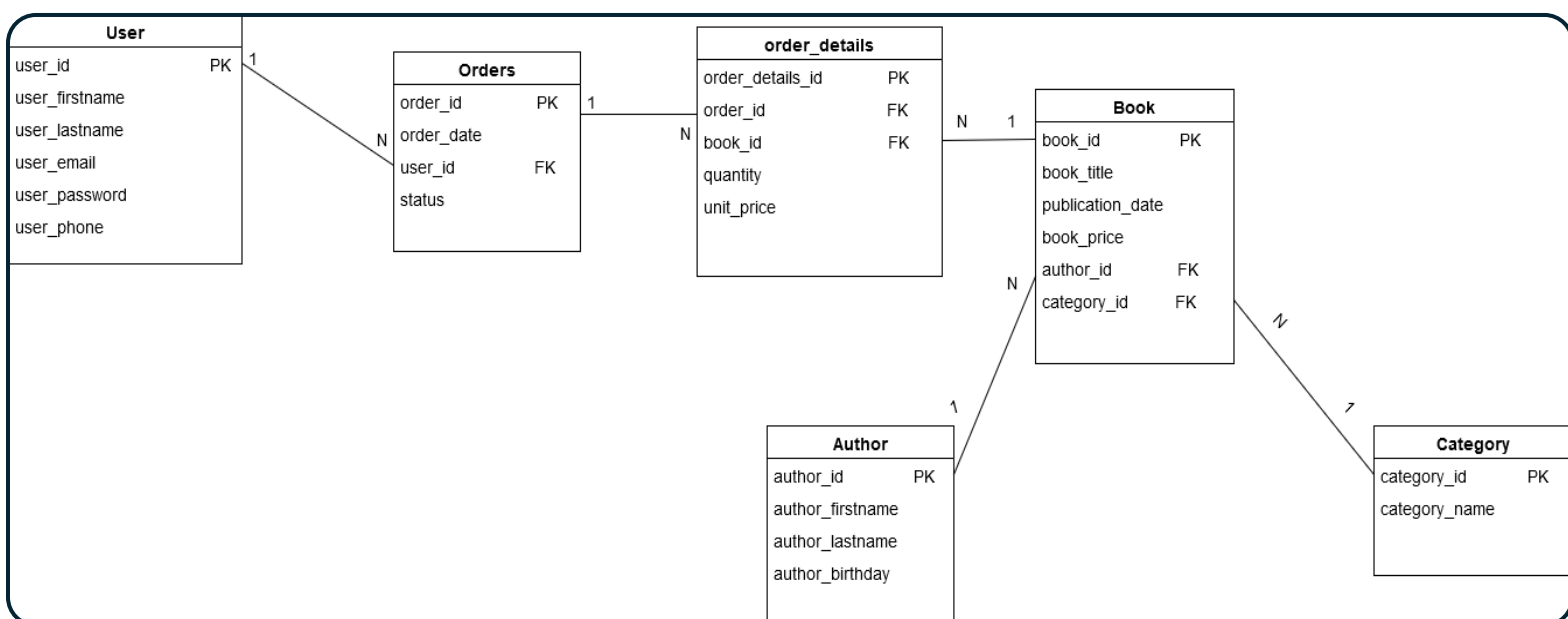


2. Modèle Logique de Données (MLD)

Le MLD traduit le MCD en **tables relationnelles** compatibles avec PostgreSQL :

- Définition des **clés primaires** (id_user, id_book, etc.) et **clés étrangères** (user_id dans Order, book_id dans Order_Details).
- Application des contraintes d'intégrité référentielle (clé étrangère avec suppression en cascade là où c'est pertinent).
- Indexation sur les colonnes souvent utilisées pour les recherches (ex. book_title, category_id).

Cette étape assure la **cohérence des données** et prépare la structure pour les opérations CRUD utilisées dans l'application.



```
User = ( user_id, user_firstname, user_lastname,  
user_email, user_password, user_phone );  
  
Orders = ( order_id, order_date, #user_id, status );  
  
Order_Details = ( order_details_id, #order_id,  
#book_id, quantity, unit_price );  
  
Book = ( book_id, book_title, publication_date,  
book_price, #author_id, #category_id );  
  
Author = ( author_id, author_firstname,  
author_lastname, author_birthdav );
```

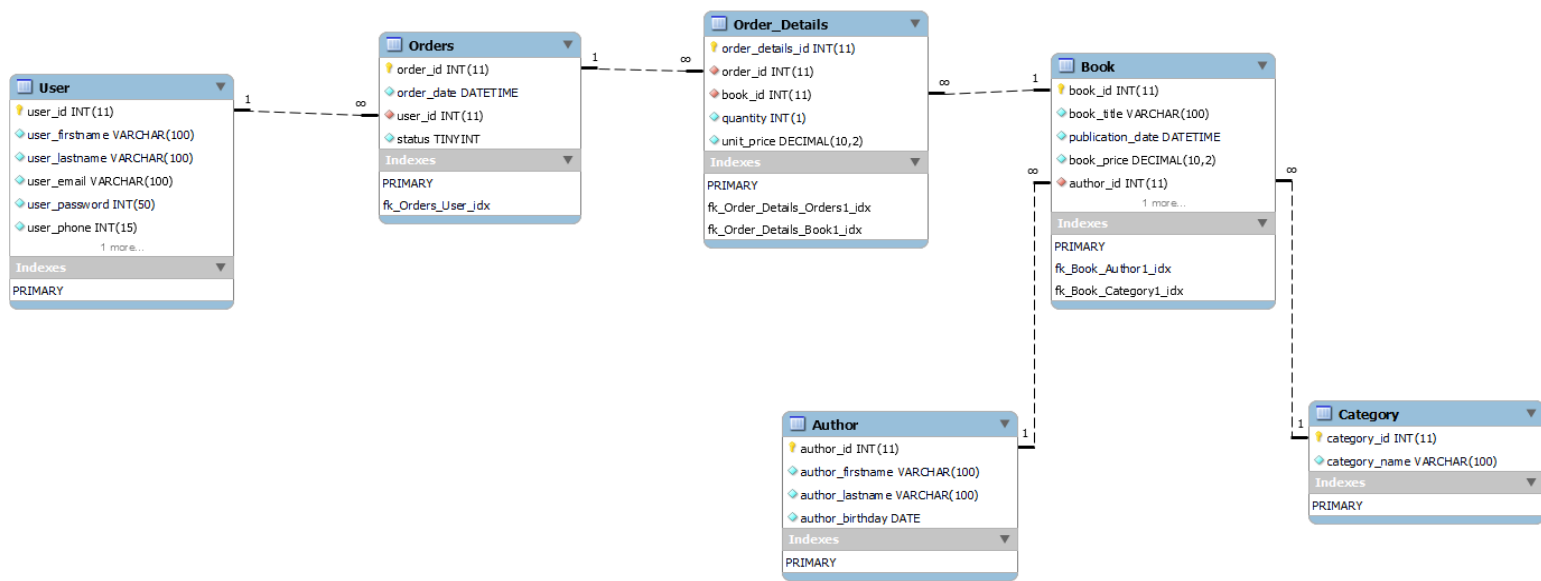
3. Modèle Physique de Données (MPD)

Le MPD reflète la structure finale dans PostgreSQL.

Exemple :

- **Table Users** : contient user_id, user_email, password_hash, role, created_at.
- **Table Orders** : enregistre order_id, user_id, total_ht, total_ttc, payment_status, created_at.
- **Table Order_Details** : relie chaque commande aux livres commandés (order_id, book_id, quantity, price).
- **Table Books** : book_id, title, author_id, category_id, price_ht, price_ttc, stock, file_path.

Ces relations garantissent un suivi complet des ventes, paiements et téléchargements.



10.4 Mettre en place une base de données

La base de données de **La Plume des Étoilées** est conçue pour stocker l'ensemble des informations nécessaires au fonctionnement du site : utilisateurs, livres, auteurs, catégories, commandes et paiements.

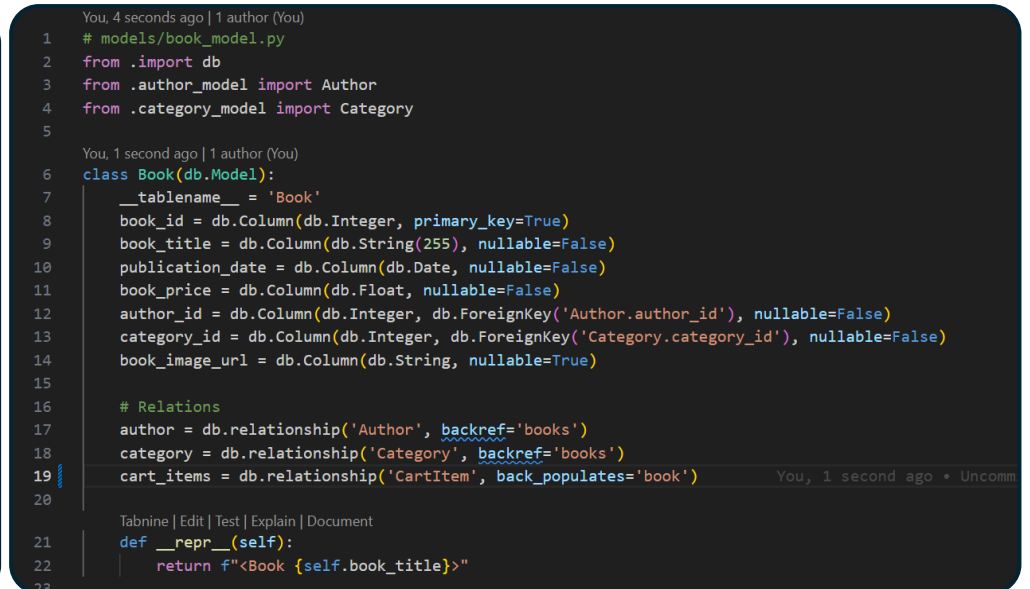
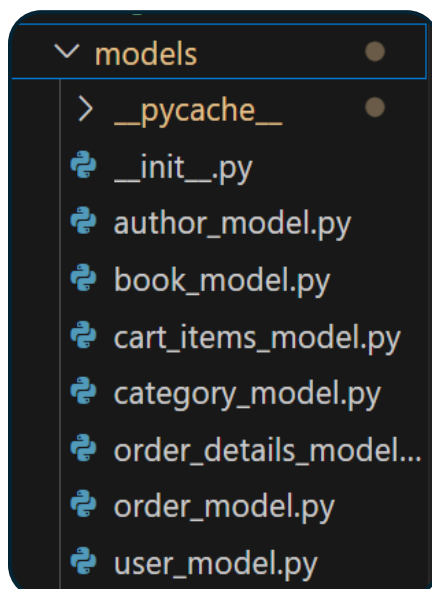
Le **Système de Gestion de Base de Données (SGBD)** retenu est **PostgreSQL**, choisi pour sa stabilité, ses performances et sa compatibilité avec l'ORM **SQLAlchemy**.

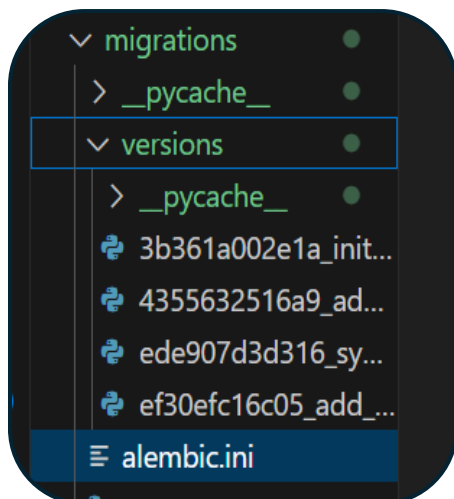
Une seconde base **MongoDB** est utilisée pour la **journalisation** des événements sensibles (logs d'authentification, paiements, actions administratives).

10.5 Création des modèles et migrations

- Les **modèles SQLAlchemy** définissent la structure des tables sous forme de classes Python (ex. User, Book, Author, Category, Order, OrderDetail).
- Chaque classe précise les **colonnes** (nom, type, contraintes), les **relations** (ForeignKey) et les **contraintes d'intégrité**.
- Une fois les modèles définis, **Alembic** est utilisé pour générer et appliquer les **migrations** :
 1. alembic revision --autogenerate → création d'un script de migration à partir des modèles.
 2. alembic upgrade head → création automatique des tables dans la base PostgreSQL.

Cette méthode garantit la **synchronisation** entre le code et la structure réelle de la base, tout en permettant un **versionnement** du schéma.





10.6 Insertion des données via l'application

Formulaire d'inscription : permet à un nouvel utilisateur de créer un compte (User). Les données sont validées côté serveur avant insertion en base.

Formulaire d'ajout de livre : réservé aux administrateurs, permet d'ajouter un nouveau livre (Book) en renseignant titre, description, prix, fichier numérique et image de couverture.

Formulaire d'ajout d'auteur : permet de créer un nouvel auteur (Author) lié à un ou plusieurs livres.

Formulaire d'ajout de catégorie : permet de classer les livres par thèmes (Category).

Chaque enregistrement est géré par une opération CRUD de type CREATE, exécutée via SQLAlchemy (`db.session.add()` + `db.session.commit()`).

10.7 Journalisation et suivi des opérations

Les événements sensibles sont stockés dans **MongoDB** :

Connexion/déconnexion utilisateur.

Cette séparation entre base métier (PostgreSQL) et base de logs (MongoDB) permet d'améliorer la **traçabilité** et de préserver les **performances**.

10.8 Sauvegardes et sécurité

Sauvegardes automatiques : un script planifié exécute quotidiennement `pg_dump` pour créer une copie complète de la base PostgreSQL.

Horodatage et rétention : chaque sauvegarde est datée et conservée selon une politique définie (rotation).

Sécurité d'accès :

- Compte applicatif PostgreSQL avec droits minimaux.
- Fichier `.env` contenant les identifiants, hors du dépôt Git.
- Accès limité au serveur via un pare-feu.

10.9 Avantages de cette mise en place

- Structure fiable et évolutive grâce à Alembic.
- Insertion de données sécurisée via des formulaires avec validation.
- Journalisation complète pour audit et sécurité.
- Protection contre la perte de données via sauvegardes régulières.

10.10 Développer des composants dans le langage d'une base de données

Dans ce projet, l'accès aux données s'appuie sur une base SQL unique hébergée sur un serveur. L'application communique avec cette base via une couche d'accès standardisée : contrôleurs Flask (Blueprints), services métier , **ORM SQLAlchemy**, PostgreSQL.

La chaîne est configurée par variables d'environnement (hôte, port, nom de base, utilisateur) et utilise un pool de connexions pour la stabilité et les performances.

Modèle et intégrité. Le schéma couvre les besoins de la boutique et du générateur d'histoires : user, book, author, category, cart_items, orders, order_details (et tables de log). Des clés étrangères garantissent la cohérence (ex. order_details.order_id → orders.order_id), des contraintes d'unicité évitent les doublons (ex. email utilisateur), et des index accélèrent les filtrages courants (catégorie, date de commande). Les mots de passe sont hachés côté serveur et jamais stockés en clair.

10.10.1 Les opérations de CRUD

CREATE, READ, UPDATE, DELETE couvrent l'ensemble des cas d'usage :

CREATE : création de compte, création de commande et de ses lignes, enregistrement des événements (paiement, génération).

```
# CREATE
@admin_bp.route('/add_book', methods=['GET', 'POST'])
@admin_required
Tabnine | Edit | Test | Explain | Document
> def add_book(): ...
```

READ : consultation du catalogue paginé, détail d'un livre, contenu du panier, historique et détail des commandes.

```
# READ
@admin_bp.route('/books', methods=['GET'])
@admin_required
Tabnine | Edit | Test | Explain | Document
> def list_books(): ...
```

UPDATE : modification d'un livre, d'un auteur ...

```
# UPDATE
@admin_bp.route('/edit_book/<int:book_id>', methods=['GET', 'POST'])
@admin_required
Tabnine | Edit | Test | Explain | Document
> def edit_book(book_id): ...
```

DELETE : Suppression d'un auteur, d'un livre ou d'une catégorie

```
# DELETE
@admin_bp.route('/delete_book/<int:book_id>', methods=['POST'])
@admin_required
Tabnine | Edit | Test | Explain | Document
> def delete_book(book_id): ...
```

Lors d'un achat, le service métier valide le panier, ouvre une transaction, vérifie les disponibilités, crée la commande (orders) puis insère les lignes (order_details). En cas d'échec paiement, la transaction est annulée (rollback) et aucun enregistrement partiel ne subsiste.

```
try:
    new_book = Book(...)
    db.session.add(new_book)
    db.session.commit()
except:
    db.session.rollback()
```

La consultation du catalogue s'effectue via des requêtes paramétrées et paginées (limitation de charge et d'I/O).

L'application utilise un compte SQL à privilèges minimaux, des requêtes préparées (via SQLAlchemy) pour neutraliser les injections). Les routes sensibles imposent un contrôle de rôle (visiteur, client, admin).

Les entrées sont échappées avant rendu, et les opérations d'écriture se font en POST avec CSRF activable selon l'environnement.

Pour améliorer la qualité et la perfs La pagination limite les volumes renvoyés et les jointures sont optimisées .

10.10.2 Journalisation NoSQL (MongoDB)

Le site enregistre les événements importants dans une base NoSQL dédiée : connexions, tentatives de paiement, actions sur le panier et erreurs applicatives. Pour chaque événement, nous conservons l'essentiel : la date et l'heure, l'utilisateur s'il est identifié, l'action réalisée, le résultat (succès/échec) et un contexte minimal (page concernée, origine de la requête). Aucune donnée sensible n'est stockée ; lorsqu'un contenu pourrait être confidentiel, nous conservons uniquement un résumé non exploitable.

Tabnine | Edit | Test | Explain | Document

```
def log_event(self,
               collection: str,
               user_id: int,
               email: str,
               event_type: str,
               metadata: Optional[Dict[str, Any]] = None) -> bool:
```

```
{
  "user_id": 123,                // Unique user identifier
  "event_type": "login",        // Event classification
  "http_metadata": {           // Request context
    "ip": "192.168.1.1",       // Client IP address
    "method": "POST",          // HTTP verb
    "path": "/auth/login"      // API endpoint
  },
  "timestamp": "2023-05-28T14:30:00Z" // Precise UTC time
}
```

Ces journaux sont gardés 90 jours puis supprimés automatiquement. Leur écriture est indépendante du parcours utilisateur : un problème de journalisation ne bloque jamais la navigation ni l'achat. L'accès aux logs est limité à l'équipe autorisée, en consultation uniquement. Ils alimentent nos tableaux de bord (p. ex. suivi des erreurs et des refus d'accès) et facilitent l'analyse en cas d'incident, tout en respectant la confidentialité des utilisateurs.

Tabnine | Edit | Test | Explain | Document

```
def purge_old_data(self, retention_days: int = 90):
    """
    Automatically purge data older than retention period
```

11 Préparer et documenter le déploiement d'une application

11.1 Repo GitHub

J'ai créé le dépôt GitHub public avec un README, et créer une branche main.
J'ai initialisé le dépôt en local, configuré mon identité Git, puis relié le distant avec `git remote add origin <URL>`.

J'ai ajouté un `.gitignore` Python/Flask (`venv/`, `.env`, `__pycache__`/, `instance/`, `.pytest_cache/`) et structuré le projet (`app/`, `models/`, `templates/`, `services/`).

J'ai réalisé le commit initial (`git add .` puis `git commit -m "chore: initial commit (structure MVC, DDL, templates)"`) en respectant une convention de message.

J'ai poussé sur main (`git branch -M main` ; `git push -u origin main`) et vérifié mes Actions/Secrets (`STRIPE_*`, `DATABASE_URL`) ainsi que le statut du workflow.

Bifurquer	Mis à jour	Vérifier le statut	Derrière	Devant	Demande d'extraction
principal	il y a 10 mois			Défaut	...
Sintique	il y a 10 mois		0	4	...
Nadège	il y a 10 mois		0	8	...

11.2 II. Préparer et documenter le déploiement d'une application

L' Objectif est de rendre l'application déployable de façon reproductible, avec une procédure claire pour local / staging / prod.

Ce qui a été fait dans le projet.

- **Architecture exécutable** : application Flask en factory (`create_app()`), Blueprints séparés (`auth_controller.py`, `cart_controller.py`, `payement_controller.py`), modèles SQLAlchemy sous `models/` (PostgreSQL).

- **Variables d'environnement** : `.env` utilisé en dev ; `.env` documente les clés requises (DB, SECRET_KEY, STRIPE_*, etc.).

```
(.venv) PS D:\DeveloppementWeb\beauvoir\Projet_git\la_plume_des_etoiles> .\.venv\Scripts\Activate.ps1
```

- **Base de données** : migrations Alembic prêtes à l'emploi (alembic/), scriptées pour init + upgrades.
- **Documents de déploiement** : un **README "Déploiement"** (ou section dédiée) décrit pas-à-pas l'installation (Python/venv), la configuration (`.env`), la migration DB et le lancement (Gunicorn/Waitress en prod).
- **Scripts utiles** : commandes regroupées (ex. flask db upgrade, commande de lancement avec flask --app "app:create_app()" run en dev).

Procédure type documentée (extrait).

1. **Pré-requis** : Python 3.x, PostgreSQL avec base et utilisateur dédiés.
2. **Config** : copier `.env.example` → `.env` et renseigner les secrets.
3. **Packages** : pip install -r requirements.txt.
4. **DB** : flask db upgrade pour appliquer les migrations.
5. **Lancement** :
 - Dev : flask --app "app:create_app()" run --debug
 - Prod (Linux) : gunicorn -w 3 'app:create_app()' -b 0.0.0.0:8000 derrière Nginx.
6. **Vérifications** : page d'accueil, création de compte, parcours panier → **/checkout-summary** → redirection Stripe (clé test).

Objectif. Intégrer la sécurité "by design" avant la mise en ligne : secrets, durcissement des en-têtes, contrôle d'accès, données et paiements.

Ce qui a été fait dans le projet.

- **Gestion des secrets** : clés (SECRET_KEY, STRIPE_SECRET, DB URI) **hors code** via `.env`.
- **Auth & sessions** : **Flask-Login** pour l'authentification, sessions sécurisées, e-mail unique à l'inscription.

- **CSRF** : formulaires protégés (token CSRF injecté côté Jinja).
- **RBAC / rôles** : séparation **utilisateur** / **admin** (accès aux pages d'administration restreint).

```
# Usage
@route('/admin/dashboard')
@role_required('admin')
def admin_dashboard():
    return render_template('admin/dashboard.html')
```

- **Mots de passe** : hachage (Werkzeug/Passlib). (Si tu as ajouté un indicateur de robustesse en front, indique-le ici.)

```
from werkzeug.security import generate_password_hash, check_password_hash

# Dans le modèle User
def set_password(self, password):
    """Hash le mot de passe avec PBKDF2-HMAC-SHA256"""
    self.user_password = generate_password_hash(
        password,
        method='pbkdf2:sha256',
        salt_length=16
    )
```

- **Stripe** : Paiement externalisé ; **aucune donnée carte** sur nos serveurs. Webhooks Stripe prévus avec **vérification de signature** (clé webhook).
- **Dépendances** : versions figées dans requirements.txt ; audit de sécurité prévu (pip-audit / Safety).

Mesures recommandées et en partie préparées.

- **PostgreSQL** : utilisateur DB à privilèges minimaux ; sauvegardes planifiées.
- **Journalisation sécurité** : traces d'authentification et d'événements sensibles (succès/échec).

12 Déploiement — GitHub Actions → Docker → Railway

Objectif est de fiabiliser le passage en prod par l'automatisation (CI/CD), la qualité (tests, le versioning et l'observabilité).

12.1 Partie GitHub — CI / CD

D'abord, je prépare l'automatisation côté GitHub. J'ajoute les secrets `RAILWAY_TOKEN` et `RAILWAY_PROJECT_ID` dans **Settings , Secrets, Actions**, afin que les workflows puissent se connecter à mon projet Railway en toute sécurité. Ensuite, je mets en place la **CI** dans `.github/workflows/ci.yml` : à chaque push ou pull request, j'installe les dépendances Python, je lance des tests rapides (smoke tests) et je construis l'image Docker pour vérifier que le build passe. Puis, je configure la **CD** dans `.github/workflows/deploy-railway.yml` : à chaque push sur main, le workflow installe le CLI Railway, se connecte avec les secrets, et exécute `railway up` pour déployer. Ainsi, je garde un flux propre : d'abord, la PR ne déclenche que la CI ; ensuite, je fusionne quand c'est vert ; puis, la CD déploie automatiquement sur Railway ; enfin, je contrôle les logs pour valider la mise en ligne.

12.2 Partie Docker

Je centralise la logique d'exécution dans une image Docker reproductible. Mon Dockerfile se trouve dans le dossier `docker/`, et j'indique à Railway de l'utiliser en définissant `RAILWAY_DOCKERFILE_PATH=docker/Dockerfile`. Ensuite, je veille à ce que l'image installe les dépendances, expose le **PORT=8000**, et lance l'application en production avec **Gunicorn** qui écoute sur `0.0.0.0:8000`. Puis, je prévois l'application des migrations au démarrage en exécutant `flask db upgrade` avant de lancer Gunicorn, afin que le schéma de la base soit toujours à jour. Ainsi, mon conteneur démarre de la même façon en local, en CI et en production, sans variations cachées ; enfin, je peux diagnostiquer un échec de build ou de démarrage simplement en lisant les logs de build ou d'exécution.


```
[+] Building 1/1
✓ la_plume_des_etoiles-web Built
→ la_plume_des_etoiles git:(main) X docker compose up -d
[+] Running 4/4
✓ Container la_plume_des_etoiles-mailpit-1 Running
✓ Container la_plume_des_etoiles-mongo-1 Running
✓ Container app_db Healthy
✓ Container la_plume_des_etoiles-web-1 Started
→ la_plume_des_etoiles git:(main) X docker compose logs -f web
web-1 | Waiting for database db_psql_labrarry at db:5432 ...
web-1 | db:5432 - accepting connections
web-1 | DB is up!
web-1 | Running migrations ...
web-1 | INFO [alembic.runtime.migration] Context impl PostgresqlImpl.
web-1 | INFO [alembic.runtime.migration] Will assume transactional DDL.
web-1 | [2025-09-11 15:01:31 +0200] [1] [INFO] Starting gunicorn 22.0.0
web-1 | [2025-09-11 15:01:31 +0200] [1] [INFO] Listening at: http://0.0.0.0:8000 (1)
web-1 | [2025-09-11 15:01:31 +0200] [1] [INFO] Using worker: gthread
web-1 | [2025-09-11 15:01:31 +0200] [9] [INFO] Booting worker with pid: 9
web-1 | [2025-09-11 15:01:31 +0200] [10] [INFO] Booting worker with pid: 10
web-1 | [2025-09-11 15:01:31 +0200] [11] [INFO] Booting worker with pid: 11
```

12.3 Partie Railway

D'abord, je crée un projet Railway avec deux éléments : un **service Web** relié à mon dépôt GitHub, et une **base PostgreSQL** ajoutée au même projet. Ensuite, je configure les **variables d'environnement** sur le service Web pour un mode production fiable : APP_ENV=production, FLASK_ENV=production, PORT=8000, TZ=Europe/Paris, une SECRET_KEY forte, et surtout DATABASE_URL pointée sur la **Reference** Postgres.DATABASE_URL. Puis, je déclenche le déploiement depuis GitHub (CD) et je surveille l'amorçage dans **Railway → Logs** : je dois voir flask db upgrade passer sans erreur, puis "Listening at 0.0.0.0:8000". Ainsi, quand tout est vert, j'ouvre l'URL <https://<mon-service>.up.railway.app> pour valider que l'application répond bien sans préciser de port. Enfin, j'active les fonctionnalités au besoin en ajoutant leurs clés (Stripe, SMTP, OpenAI, Mongo Atlas) comme variables Railway, et je gère les imprévus en **rollback** : soit je git revert vers un commit stable puis je pousse sur main pour redéployer automatiquement, soit je redéploie une version antérieure depuis l'historique des builds dans l'interface Railway.

 la_plume_des_etoilees

Deployments

Variables

Metrics

Settings

22 Service Variables

Shared Variable

Raw Editor

New Variable

> 8 Railway Provided Variables available

{ APP_ENV	*****	:
{ CANCEL_URL	*****	:
{ FLASK_APP	*****	:
{ FLASK_ENV	*****	:
{ MAIL_DEFAULT_SENDER	*****	:
{ MAIL_PASSWORD	*****	:
{ MAIL_PORT	*****	:
{ MAIL_SERVER	*****	:
{ MAIL_USE_SSL	*****	:
{ MAIL_USE_TLS	*****	:
{ MAIL_USERNAME	*****	:
{ MONGO_URI	*****	:
{ MONGODB_DB	*****	:
{ ODMAT_APP_KEY	*****	:

Remerciements

Je tiens à exprimer toute ma gratitude à l'École Beauvoir pour son accompagnement, et plus particulièrement à **Stéphanie Pottecher** et **Guillaume Chemin** pour leur disponibilité, leurs conseils et leur soutien tout au long de mon parcours.

Je remercie également ma famille pour son appui constant, sa patience et ses encouragements, qui ont été essentiels à la réussite de ce projet.

Conclusion

En conclusion, ce projet m'a permis de développer des compétences techniques et méthodologiques solides, tout en approfondissant ma capacité à travailler de manière structurée et professionnelle. Au-delà de l'aspect technique, j'ai pu mettre en pratique une démarche complète allant de l'analyse des besoins jusqu'au déploiement, en intégrant des critères de sécurité, de qualité et d'accessibilité. Cette expérience a renforcé ma motivation à poursuivre dans le domaine du développement et de la data, avec une vision claire des enjeux et des bonnes pratiques à adopter dans un contexte professionnel.